

CN Assignment

Protocol Specification Document

1. Overview

This document specifies a custom application-layer protocol, referred to as the file transfer protocol, built on top of TCP sockets. The protocol supports user authentication, private and broadcast messaging, direct file upload/download to the server, and file transfer between clients mediated by the server. The server is multi-threaded and supports concurrent client sessions.

2. Transport and Message Framing

All communication occurs over TCP. Since TCP delivers a byte stream with no inherent message boundaries, it uses two framing strategies depending on content type:

Control commands (text): newline-terminated ASCII strings, encoded UTF-8. Fields are separated by single spaces. Every command line ends in `\n`.

File payloads (binary): every command that transfers a file first sends the file size in bytes as part of the command line. The receiver then performs a length-bounded read loop, calling `recv()` repeatedly and accumulating bytes until exactly size bytes have been read (a single `recv()` call is not guaranteed to return the full payload).

No readiness handshake. The sender does not wait for an acknowledgement before streaming file bytes - it sends the header line and immediately begins writing the payload. This means the receiving side must be prepared to switch from reading a text command to reading exactly `<size>` raw bytes the moment it finishes parsing a file-transfer header, with no negotiation step in between.

3. Command Reference

Commands are sent Client -> Server unless otherwise noted. Responses follow the format described in Section 5.

Command	Direction	Purpose
LOGIN <username>	C -> S	Authenticate with a unique username. No password in this iteration (per assignment scope).
LOGOUT	C -> S	Graceful disconnect; server removes user from active-users map.
LIST	C -> S	Request the list of currently active usernames.
MSG <user> <text>	C -> S -> C	Private message; server routes to the named user if online.
BROADCAST <text>	C -> S -> C*	Message delivered to every currently connected client.
UPLOAD <filename> <size>	C -> S	Store a file of <size> bytes in the sender's server-side storage.
DOWNLOAD <filename>	C -> S	Retrieve a previously uploaded file from the server.
SEND <user> <filename> <size>	C -> S -> C	Transfer a file to another user via the server (see Section 6).
INCOMING <user> <filename> <size>	S -> C	Server push notifying a client that a file follows (from SEND or pending delivery).

5. Server Response Format

Every command produces exactly one immediate response line from the server, in addition to any asynchronous pushes (e.g. INCOMING, forwarded MSG/BROADCAST). Two response types are defined:

OK <info...>
ERR <code> <reason>

Code	Meaning
001	USERNAME_TAKEN - login rejected, username already active
002	FILE_NOT_FOUND - requested file does not exist on server
003	USER_OFFLINE - target user for MSG/SEND is not currently connected and unknown to the server this session
004	MALFORMED_COMMAND - command could not be parsed
005	UNAUTHENTICATED - command received before a successful LOGIN

6. SEND Semantics: Offline-Target Delivery

Unlike MSG (which is rejected outright if the target is offline), SEND guarantees eventual delivery. The server maintains a PendingTransfers dictionary mapping username -> list of {Sender, FileName, FilePath, FileSize} entries, guarded by StateLock (the same lock used for the active-users map; see Section 8).

- **Bytes are always received from the sender first.** Regardless of whether the target is online, the server must consume exactly <size> bytes from the sender's socket immediately, since the sender begins streaming right after its command line. Skipping this would desynchronize the sender's connection (their next command would be misread as leftover file bytes).
- **Target online:** the received bytes are staged to Storage/.send_temp/<sender>_<filename>, then immediately forwarded to the target's live socket as an INCOMING <sender> <filename> <size> push followed by the file bytes, both sent while holding the target's SendLocks entry so the header and payload stay atomic. The sender receives OK SEND <target> <filename> and the temp file is deleted.
- **Target offline:** the received bytes are written directly to Storage/pending/<target>/<filename>, and an entry is appended to PendingTransfers[<target>] under StateLock. The sender receives OK SEND_QUEUED <target> <filename>.
- **Delivery on login:** immediately after a successful LOGIN handshake and before the client's command loop begins, the server pops that username's list from PendingTransfers (a snapshot taken under StateLock, then released before any network I/O). For each queued entry it streams INCOMING <sender> <filename> <size> followed by the file's bytes (read back off disk), then deletes the pending file.

Pending storage (Storage/pending/) is kept separate from a user's own UPLOAD/DOWNLOAD directory (Storage/<username>/) and from the transient SEND staging area (Storage/.send_temp/) to avoid filename collisions between a user's own files and files queued for or in transit to them.

7. Sequence Diagrams

Sequence 1: LOGIN (success and duplicate-username rejection)

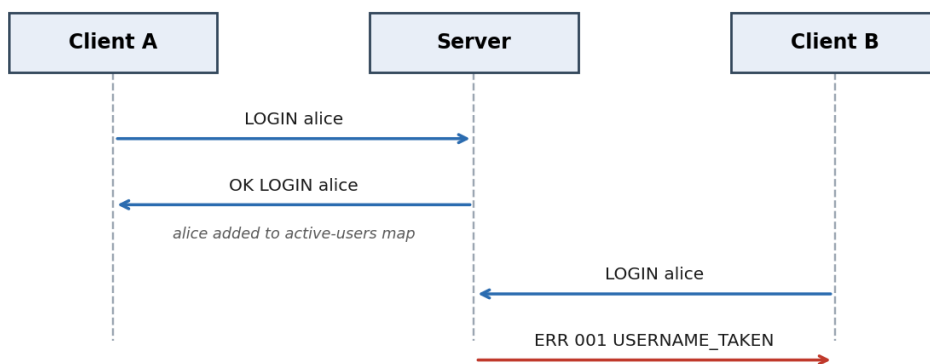


Figure 1. LOGIN: successful authentication, followed by a rejected duplicate-username attempt.

Sequence 2: LOGOUT

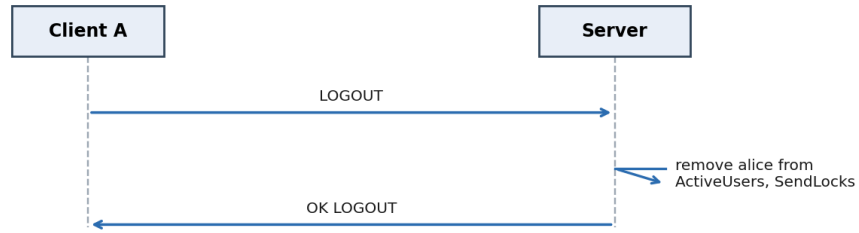


Figure 2. LOGOUT.

Sequence 3: LIST

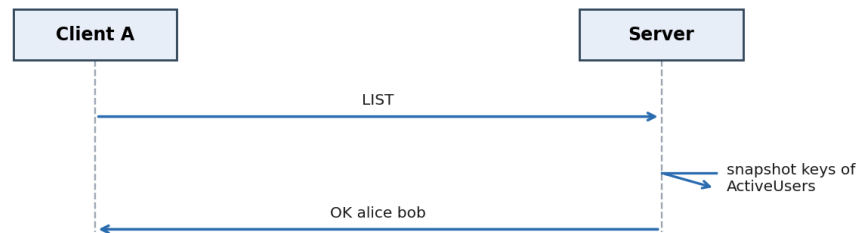


Figure 3. LIST.

Sequence 4: Private MSG between two online users

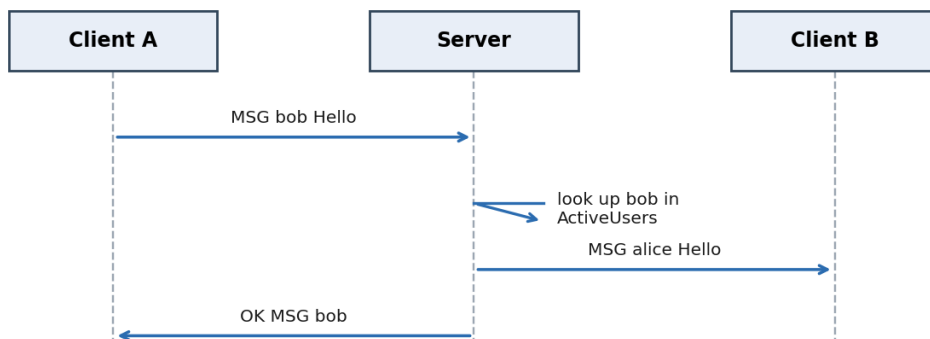


Figure 4. Private MSG delivered between two online clients via server-side routing.

Sequence 5: BROADCAST to all connected clients

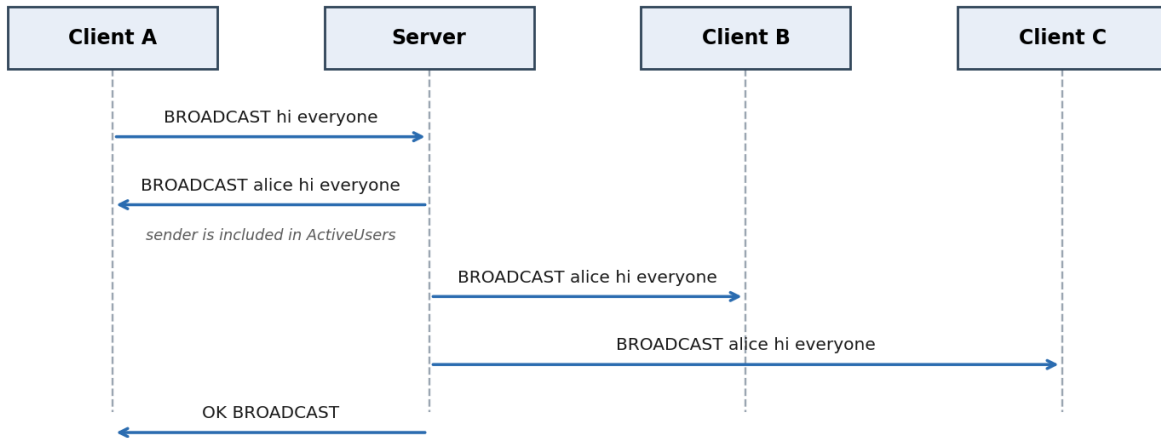


Figure 5. BROADCAST delivered to every connected client, including the sender.

Sequence 6: SEND - target user online (live forward)

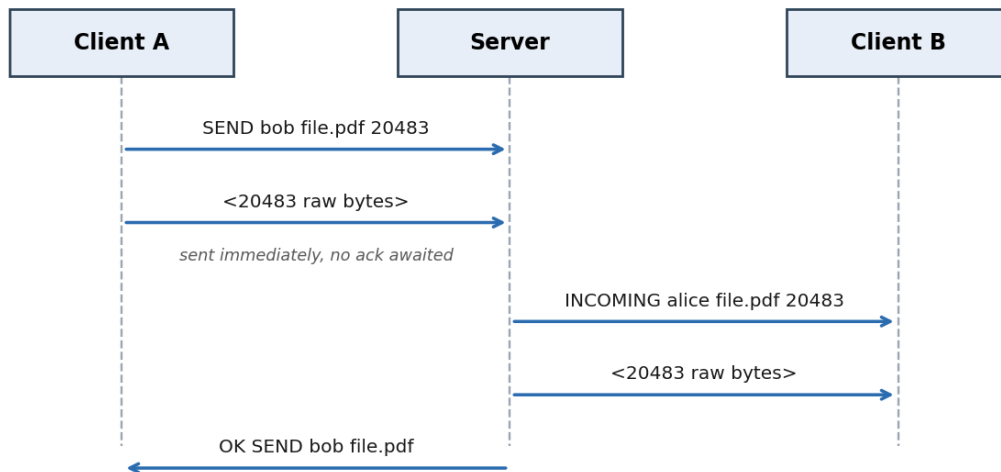


Figure 6. SEND when the target user is online: server forwards the file live.

Sequence 7: SEND - target offline, auto-delivered on next LOGIN

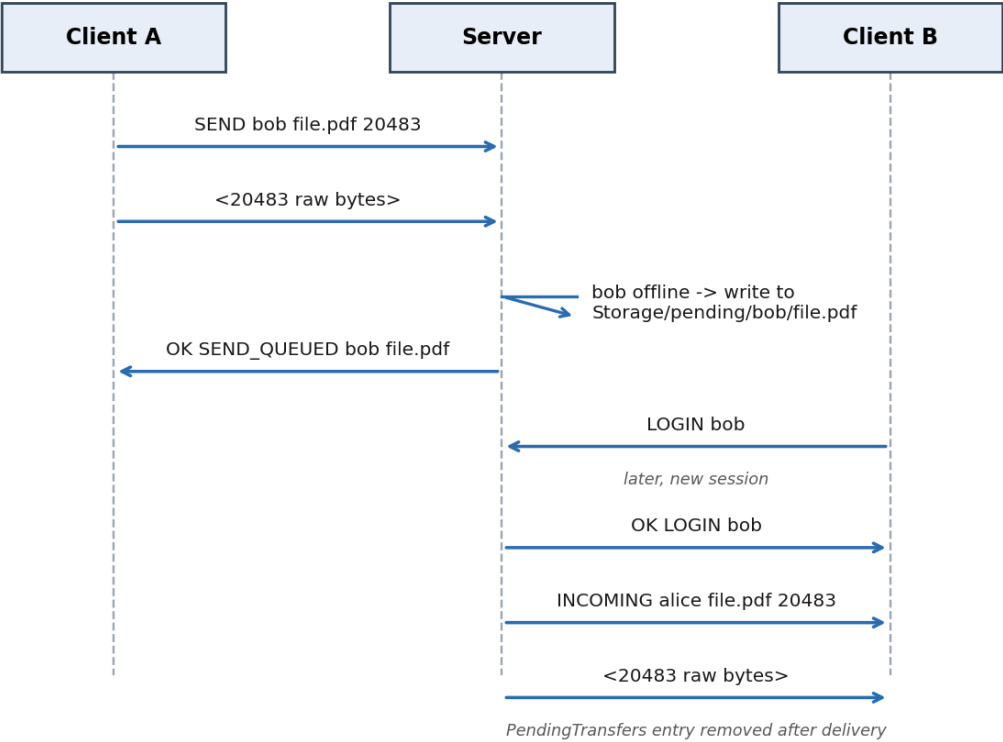


Figure 7. SEND when the target is offline: file is queued and auto-delivered on the target's next LOGIN.

Sequence 8: UPLOAD and DOWNLOAD

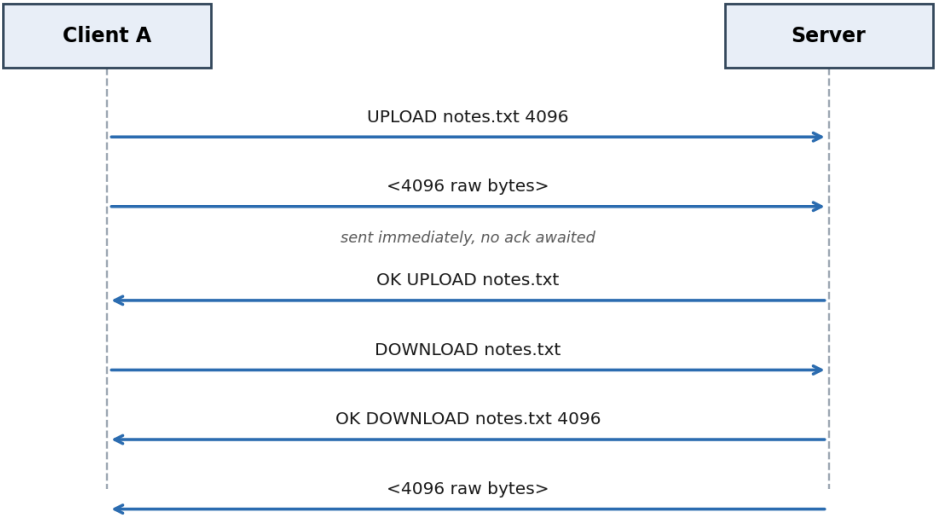


Figure 8. UPLOAD followed by DOWNLOAD of the same file.